

1. Stack Derived From Vector**04A**

讲义中实现的Stack，是在底层引入一个Vector，并将其Front/Rear指定为Bottom/Top。

- a) 试**颠倒**Front/Rear的角色，仿照示例代码实现另一个Stack；
- b) 如此改动之后，Stack各接口的**性能**有何变化？为什么？

2. Stack Derived From List**04A**

其实在示例代码中，也已基于List实现了一个Stack。

- a) 试在示例代码包中找到对应的项目，确认List之Front/Rear在其中的角色分工；
- b) 如果**颠倒**二者的角色后再实现Stack，各接口的**性能**有何变化？为什么？

3. Theseus' Tools**04B1**

如果将程序执行过程中的调用跟踪图比作Theseus探索的迷宫，那么调用栈就对应于他所使用的那个线团。然而为了避免“故地重游”甚至“原地转圈”，他还需要**更多工具**来标记已访问过的位置，比如一支粉笔。在C/C++等编程语言中，是否也对应地有类似的机制？

4. Recursion Depth & Storage Cost**04B1**

对于采用分治策略的算法，如果通过优化使得调用栈的每一帧都能节省**一半**空间，那么可求解问题的规模将会扩大到何等程度？如果能节省到**1/10**呢？（估计时可暂忽略其他方面的空间消耗。）

5. Recursion & Iteration**04B2**

- a) 是否从理论上讲，程序中的**任何**递归都可消除掉？
- b) 如果不是，原因何在？或者，试举一例；
- c) 如果总是可以，消除递归的通用方法是什么？

6. Binary/Linear Recursion**04B2**

分治算法通常都对应于二分递归，每个递归实例都含有一前一后两次递归。如果后一次递归符合**尾递归**的特征——在每个递归实例中都是最后一步操作——那么是否也可套用讲义中的方法将其消除，从而先把算法**转化**为线性递归的模式？

7. Almost-Tail Recursion**04B2**

讲义中给出的fac()算法，尽管按照定义还不是**严格**的尾递归，却非常接近——在递归语句之后，并无实质的计算——故不妨称之为**准**尾递归。

- a) 试重写该算法，要求彻底甩掉栈，将空间复杂度优化至 $\mathcal{O}(1)$ ；
- b) 你针对fac()的去递归的方法，可否推广到一般的**准**尾递归算法？

8. Head Recursion**04B3**

细察本节针对进制转换给出的那个递归算法。

- a) 在递归调用之前它其实做了一次除法，既如此，还能说它是**头递归**吗？
- b) 为什么说，**纯粹**的头递归（在递归基之后便直接递归，不做任何操作）必然会导致调用栈的**溢出**？
- c) 为什么说对于头递归，调用栈记录的信息中，只有调用参数和输出才是必要的？

9. Destruct A Singly Linked List**03B3**

考查所谓的**单向链表**：节点之间只有succ引用，而没有pred引用。

- a) 试实现一个**头递归**算法，在线性时间内完成单向链表的**析构**；
- b) 你的算法，可能是**纯粹**的头递归吗？
- c) 将你的算法改写为迭代式的，时间复杂度不变；
- d) 相对于递归版本，迭代版本在空间成本方面有多大的优化？

10. Customized Parenthesis**04C**

试拓展讲义中的括号匹配检测算法，使之能够：

- a) 支持（如Html、Jason等格式中）可自定义的**任何**“括号”；
- b) 在判断出失配的同时，还指出错误出现的**位置**，并给出**修正**的建议。

11. Priority Table**04D**

考查本节所给的运算符优先级表：

- a) 基于这张查询表，evaluate()算法对于“ 2^3^4 ”是如何处理的？
- b) 如果要与**数学课**上默认的优先级一致，这张表应该如何调整？算法同时还需如何调整？
- c) 如此调整之后，**时间**、**空间**复杂度有何变化？
- d) 该表中有若干项都是‘ ’，它们分别在什么情况下会起**作用**？

12. calcu()**04D**

当optr栈顶运算符优先于当前运算符时，evaluate()算法会调用calcu(opnd, optr.pop())执行计算。

- a) 试实现这个函数；
- b) 你的calcu()如何区分一元运算与二元运算？
- c) 对于不满足交换律的减法、除法，你的calcu()也能正确执行计算吗？
- d) 你的calcu()如何处置除零的情况？

13. Stack Size In evaluate()**04D**

- a) 试验证：在evaluate()算法中的任何时刻，optr栈中相邻运算符的相对优先级，都是完全**一致**的。
- b) 试举例说明：上述一致性，并不满足**传递律**。
- c) 既然运算符种类**有限**，optr栈及opnd栈规模的上限是否 $O(1)$ ？
- c) 试对于任意的 $n \gg 2$ 构造一个表达式，使得evaluate()算法中opnd栈的规模可以达到 n 。

14. RPN Convert**04E**

- a) 试证明：按照讲义中介绍的手工方法，的确可将任意的常规表达式转换为RPN；
- b) 试证明：evaluate()算法转换RPN的算法，也是正确的；
- c) 尽管最终的效果相同，这两种算法的转换过程有何区别？

15. RPN**04E**

我们在讲义中看到，RPN的求值算法及其过程，都要比对应的常规表达式更加简明、高效。然而我们也注意到，无论是手工还是自动地完成后者到前者的转换，本身也需要线性的时间。既如此，RPN的**价值**何在？

16. Polish Notation**04E**

如果采用PN（运算符均前置而非后置）来做表达式求值，是否也能如RPN那样同样地简明、高效？为什么？

17. Stack Permutation: Forbidden Patterns**04F**

- a) 试证明：含有 "...k...i...j..." 禁形的任何序列，必然**不是**栈混洗序列；
- b) 试证明：含有 "...j+1...i...j..." 禁形的任何序列，必然**不是**栈混洗序列；

18. Stack Permutation: Yet Another Counting**04F**

关于栈混洗的计数，讲义中所给出的递推式，是根据“A**栈顶**元素何时**进入**B栈”来做分类。

- a) 试变换视角，依据“A栈中哪个元素**最后进入**B栈”来做分类，从而导出另一个递推式。
- b) 改成看“A栈中哪个元素**率先进入**B栈”呢？

19. CSTDSA**04F**

考查栈<CSTDSA>的**混洗**序列。其中两个字符s，不妨以s_a和s_b区别。

- a) 试验证：形如[*****s_a>的混洗序列必然形如[C*****s_a>;
- b) 形如[C*****s_a>的混洗序列共有多少个？
- c) 试验证：形如[*****s_b>的混洗序列必然形如[*****AS_b>;
- d) 形如[*****AS_b>的混洗序列共有多少个？
- e) 形如[C***AS_a>和[C***AS_b>的混洗序列，各有多少个？
- f) 形如[C***AS_a>和[C***AS_b>的混洗序列之间，**不同、相同**的各有多少个？
- f) 形如[*****s>混洗序列，共有多少个？

20. Stack Permutation: Identification**04F**

本节给出了一个算法，通过**模拟**来甄别栈混洗序列。

试证明该算法是正确的：它返回的r小于n，当且仅当B不是栈混洗序列。

21. Stack Permutation vs. Parenthesis**04F**

- a) 按照讲义及课上讲解的思路严格证明： n 对括号构成的合法表达式，与 n 个元素的栈混洗——**对应**；
- b) 你是否还能从**其他**角度，理解并证明这一对应关系？

22. Parenthesis Expression**04F**

不难理解，由 n 对括号构成的所有合法表达式，可按字典序从1到Catalan(n)编号。

试阅读D. E. Knuth的The Art Of Computer Programming第4卷第4册，了解以下问题及算法：

- a) Algorithm P (I. Semba, 1981): 按字典序枚举由 n 对匹配括号组成的所有表达式；
- b) Algorithm U (F. Ruskey, 1978): 按字典序取出其中的第 k 个；
- c) Algorithm W (D. B. Arnold & M. R. Sleep, 1980): 等概率地从中随机选出一个。

23. Queue Derived From Vector**04G**

- a) 讲义中选择了基于**列表**来实现队列，是否也可改为基于**向量**来实现？
- b) 如果可以，具体如何实现？
- c) 如果不可以，原因何在？

24. More Queue Applications**04H**

除了讲义中所举的实例，你还知道在哪些计算过程中使用了这种结构？

25. Maximum Rectangle In A Histogram**04I**

- a) Brute-force算法在**最坏**情况下需要运行多少时间？试举出这样的实例；
- b) 不难理解，最大矩形可能同时存在**多个**。为此，讲义中**约定**着眼于其中的最靠左（leftmost）者。试给出这个“最靠左”的具体定义？
- c) 讲义中蛮力、两趟扫描、单趟扫描算法所返回的，是否与上述**约定**吻合？Excel演示程序呢？

26. 2-Pass vs. 1-Pass**04J**

本节针对直方图内最大矩形问题不仅介绍了蛮力算法，而且进而介绍了两趟扫描（2-Pass Scan）算法以及单趟扫描（1-Pass Scan）算法。

- a) 单趟扫描算法所具有的特性中，有哪些是两趟扫描算法并不具备的？
- b) 试分别运用04L节中的三种典型方法，分析两趟扫描、单趟扫描算法的**均摊**复杂度；
- c) 就**最坏**情况而言，三个算法的空间复杂度各是多少？
- d) 就**最好**情况而言，三个算法的空间复杂度各是多少？
- e) 就**平均**情况而言，三个算法的空间复杂度各是多少？

27. Max Queue**04J**

试分别采用04L节中介绍的方法，**分析**Max Queue结构的**均摊**复杂度。

28. Stacks As Queue**04K**

讲义中针对“双栈当队”策略的均摊复杂度，介绍了三种典型的分析方法。然而从**字面**上看，它们所得复杂度的**常系数**还是有所区别。这种区别仅仅是常系数的放缩，还是**本质**的？